

Autonomous Spark SQL Optimization Agent Design: Comprehensive Context Architecture and Decision-Making Frameworks

Introduction to the Distributed Optimization Paradigm

The modern data lakehouse architecture represents a monumental shift in how organizations process, store, and analyze petabyte-scale datasets. At the core of this ecosystem sits Apache Spark, a distributed computation engine that relies on its Catalyst Optimizer to translate high-level declarative SQL or DataFrame transformations into highly optimized physical execution Directed Acyclic Graphs (DAGs).¹ The Catalyst engine operates by parsing these operations into an unresolved logical plan, resolving it against a catalog, and applying a rigorous series of rule-based and cost-based optimizations before producing physical execution code via the Tungsten engine.¹ Within this pipeline, the Cost-Based Optimizer (CBO) is the central nervous system responsible for navigating the mathematically vast search space of physical join orderings, filter pushdowns, and broadcast thresholds.²

To model the computational cost of each candidate plan effectively, the CBO relies on a steady stream of structural metadata, specifically table-level and column-level statistics.³ Table-level statistics, such as total row counts and raw byte sizes, provide the optimizer with a baseline scale sense, enabling decisions regarding network broadcasting.³ Column-level statistics introduce a much higher degree of granularity. By evaluating approximate distinct counts generated via HyperLogLog algorithms, min/max range values, null occurrence counts, average column lengths, and sophisticated height-balanced histograms, the CBO can estimate intermediate result sizes, filter selectivity, and join fanouts across massive distributed networks.³

However, in environments characterized by continuous data ingestion, rapid state mutation, and hybrid storage formats like Delta Lake and Apache Iceberg, these static statistics decay at an accelerating rate.⁴ When statistics become stale, the Catalyst optimizer falls back on inaccurate file-size heuristics.⁴ Because Parquet files are heavily compressed, relying purely on on-disk byte sizes routinely causes the optimizer to underestimate in-memory data volumes by factors of two to ten.⁴ These estimation errors compound multiplicatively across complex query plans, leading to catastrophic execution failures such as Out-Of-Memory (OOM) errors on the Spark driver, catastrophic network shuffle degradation, and suboptimal join strategies.⁴ While the introduction of Adaptive Query Execution (AQE) in Spark 3.0 provided a vital safety net by allowing the engine to re-optimize physical plans using runtime shuffle statistics⁶, AQE is fundamentally reactive. It cannot alter the initial stage scheduling, nor can it bypass the mandatory first-stage network shuffles required to generate its runtime metrics.⁶

This architectural analysis details the blueprints for an autonomous artificial intelligence agent explicitly designed to bridge the chasm between static CBO planning and dynamic AQE runtime adjustments. Operating as a continuous, background control loop, the agent autonomously prescribes targeted database statistics instructions—such as ANALYZE TABLE, COMPUTE STATISTICS, and intelligent metadata log synchronization—to maintain optimal CBO health. The design focuses exhaustively on three pillars: the mathematical Cost-Benefit Arbitration Matrix governing intervention, the telemetry systems tracking State Drifting, and the Execution Safety Guardrails required to prevent infrastructure denial-of-service events during metadata operations.

1. The Cost-Benefit Arbitration Matrix

The core intervention mechanism of the autonomous agent involves dispatching high-overhead metadata computation queries to the underlying compute clusters. Calculating exhaustive column-level statistics carries a substantial CPU and I/O penalty.³ Extracting distinct counts and generating height-balanced histograms over thousands of compressed Parquet files fundamentally requires full or partial data scans, taxing the object storage API limits and consuming executor cores that could otherwise serve primary workloads.³ Therefore, the agent must utilize a deterministic mathematical framework to weigh the infrastructural cost of executing an ANALYZE command against the projected reduction in downstream query execution latency.

1.1 Deterministic Mathematical Scoring Formula

The Cost-Benefit Arbitration Matrix operates within a Theoretically Sound Multi-Objective Optimization (MOO) framework.⁷ The framework evaluates the dual objectives of minimizing compute latency and restricting cloud resource expenditure. The agent determines whether to trigger statistics gathering based on an intervention score $I(T)$, calculated continuously for a given table T .

Let $Q = \{q_1, q_2, \dots, q_n\}$ represent the set of historical and predicted future queries

referencing table T . The agent parses historical execution logs to build this probability distribution.

Let $P(q_i)$ represent the probability or execution frequency of query q_i executing within the next temporal window Δt .

Let $C_{\text{default}}(q_i)$ represent the estimated execution cost of the query under the current, potentially decayed catalog statistics.

Let $C_{\text{optimal}}(q_i)$ represent the theoretical query execution cost if perfect, up-to-date column statistics were available to the Catalyst planner.

Let $C_{\text{analyze}}(T)$ represent the computational and financial cost of running the ANALYZE TABLE COMPUTE STATISTICS job on table T .

Let $R(T)$ represent the infrastructural risk factor of the operation, scaling from 1.0 to a severe penalty multiplier.

The intervention score $I(T)$ is calculated using the following deterministic equation:

$$I(T) = \left(\sum_{i=1}^n P(q_i) \cdot [C_{\text{default}}(q_i) - C_{\text{optimal}}(q_i)] \right) - (C_{\text{analyze}}(T) \cdot R(T))$$

If $I(T) > \tau$, where τ is a tunable environmental intervention threshold managed by the platform administrators, the agent autonomously queues the statistics gathering command. The Spark CBO estimates the cost of a join operator using a relatively simple but effective internal formula: $\text{cost} = \text{weight} \cdot \text{cardinality} + (1.0 - \text{weight}) \cdot \text{size}$.⁹ The first portion of this cost roughly corresponds to the CPU overhead required to hash and probe records, while the second portion corresponds to the I/O cost of shuffling data across the network.⁹ The agent leverages this internal Catalyst formulation to simulate the differential between C_{default} and C_{optimal} . The projected savings fundamentally derive from the CBO's ability to minimize total intermediate data flow by accurately enumerating candidate join orderings and picking the route that minimizes intermediate matrix sizes across deeply nested execution trees.³ By simulating the output rows as $\text{output_rows} \approx (\text{left_rows} \times \text{right_rows}) / \max(\text{left_distinct_count}, \text{right_distinct_count})$ ³, the agent can calculate the exact byte-level savings a fresh statistics update will yield on the network shuffle phase.

1.2 Quantifying the Risk of Expensive Column-Level Scans

Running a full column-level scan on a highly voluminous table containing hundreds of columns poses severe systemic risks. Data lake formats require Spark to open files, read footers, check row group statistics, and execute heavy aggregations, leading to immense object storage API overhead.¹⁰

The risk factor $R(T)$ acts as the critical dampener in the arbitration matrix. It is modeled as a function of total data volume (V), schema width (W), cardinality complexity (K), and target file count (F):

$$R(T) = \alpha \cdot \left(\frac{V}{B_{\text{network}}} \right) + \beta \cdot (F \cdot L_{\text{API}}) + \gamma \cdot \left(\sum_{j=1}^W \mathcal{O}_{\text{stats}}(C_j, K_j) \right)$$

Where:

- B_{network} is the provisioned network bandwidth of the underlying compute infrastructure.
- L_{API} is the latency of the object storage API (e.g., S3 GET requests required to read Parquet footers).
- $\mathcal{O}_{\text{stats}}$ represents the complexity of computing statistics for column C_j given its cardinality K_j . Collecting statistics on columns containing long string or binary values is exponentially more expensive than numeric types.⁸
- α, β, γ are infrastructure-specific tuning constants derived from the environment's compute budget context.

To mitigate this risk programmatically, the agent actively manipulates Delta Lake and Iceberg table properties to restrict the scope of the ANALYZE operation. By default, Delta Lake restricts automatic statistics collection to the first 32 columns of a table's schema, as dictated by the `delta.dataSkippingNumIndexedCols` property.⁸ However, this positional heuristic is often flawed, as critical join keys might reside later in the schema.

Rather than inflating the overall numeric limit and ballooning the write overhead and transaction log size, the agent leverages historical query profiles to identify the top 10 to 20 most frequently filtered columns.¹¹ It then dynamically injects the `delta.dataSkippingStatsColumns` property, explicitly naming the target columns to be tracked.¹¹ By overriding the positional default, the agent surgically reduces the schema width variable (W) in the risk equation, driving $R(T)$ down to acceptable levels and allowing the arbitration matrix to proceed with the update.

1.3 Boundary Rules: AQE Runtime Corrections vs. Static CBO Pre-Computation

A paramount architectural decision in the agent's logic is knowing precisely when to delegate optimization entirely to Adaptive Query Execution, and when static CBO intervention is strictly required. AQE dynamically handles shuffle partition coalescing, skew join optimization, and Broadcast Hash Join (BHJ) conversion.⁶ However, AQE functions exclusively *after* stage boundaries are hit and data has been written to local disk.⁶

The agent operates on a strict set of deterministic boundary rules to adjudicate between static CBO injection and deferred AQE optimization.

Execution Context	Agent Decision	Architectural Rationale
Table Size Proximate to autoBroadcastJoinThreshold	Force Static CBO (Explicit ANALYZE)	If a table's actual in-memory size is slightly below the 30MB default broadcast threshold ⁶ , AQE will still force the engine to shuffle the data before it recognizes the table is small enough to broadcast. Pre-computing stats allows the Catalyst planner to apply an explicit /*+ BROADCAST(t) */ hint statically. ⁴ This entirely skips the network shuffle phase, completely bypassing the disk-write exchange that AQE relies upon to make its decision. ⁴
High Data Skew on Rapid Mutation Streams	Defer to AQE	Computing static histograms on rapidly streaming, heavily skewed datasets guarantees immediate staleness. AQE's skewed join optimization detects block sizes post-shuffle and automatically splits skewed partitions across multiple tasks, making static histograms redundant and computationally wasteful. The agent sets spark.sql.cbo.enabled=false for these specific queries to save compute cycles. ¹²
Deep Multi-Way Star Schema Joins	Force Static CBO	AQE cannot globally reorder deeply nested,

		multi-table join DAGs once stage execution begins. The CBO must enumerate candidate join orderings based on intermediate result sizes ($\text{left_rows} \times \text{right_rows} / \max(\text{left_distinct}, \text{right_distinct})$)³ before the first execution node is triggered. If the CBO is starved of stats here, it may execute massive Cartesian products early in the DAG, causing unrecoverable driver OOMs.⁴
Complex Expressions and UDFs in Join Keys	Defer to AQE	The Catalyst optimizer fundamentally cannot estimate selectivity for complex User Defined Functions (UDFs) or non-standard custom logic.⁴ The agent recognizes this limitation through logical plan parsing, avoids wasting compute on static stats that will be ignored by Catalyst, and relies exclusively on AQE runtime metrics to salvage the physical plan.
High Query Concurrency in Shared Environments	Force Static CBO	In shared analytical clusters processing TPC-DS-style workloads, AQE's reliance on initial shuffle phases consumes vast amounts of cluster memory simultaneously. Pre-computing statistics

		centrally prevents hundreds of concurrent queries from experiencing "first-stage" resource bloat, ensuring stability through proper sizing and minimizing query latency. ¹²
--	--	--

By codifying these architectural boundaries, the agent avoids competing with the native AQE engine. Instead, it creates a symbiotic environment where AQE operates strictly on edge cases and runtime anomalies that the CBO cannot resolve mathematically.

2. State Drifting & Temporal Telemetry

In hybrid lakehouse architectures, data is rarely static. Enterprise tables undergo continuous mutation via high-velocity appends, targeted updates, and regulatory deletes.⁵ As the underlying physical storage state drifts from the catalog's recorded metadata state, the optimizer's statistics decay rapidly. A foundational component of the autonomous agent is a sophisticated telemetry system that tracks table mutation velocity and calculates precise statistical decay thresholds without invoking heavy compute workloads.

2.1 Metadata Telemetry and Mutation Velocity Tracking

To circumvent the immense computational cost of scanning physical Parquet files to determine state drift, the agent interacts directly with the transactional metadata layers of modern open table formats, primarily Delta Lake, Apache Iceberg, and Apache Hudi.⁵ Delta Lake, for instance, maintains a transaction log (`_delta_log`) comprising sequentially numbered JSON commit files and periodic Parquet checkpoint snapshots.⁵ This transaction log serves as the single source of truth for the table's state, housing file-level column statistics, partition definitions, and schema evolutions.⁵

The agent utilizes lightweight, non-Spark libraries—such as `delta-rs` (a Rust implementation of the Delta protocol with Python bindings) or the DuckDB Delta Kernel extension—to parse the `AddFile` and `RemoveFile` actions directly from the object store.¹⁴ By evaluating these active add actions, the agent can retrieve the exact partition values, record counts, and min/max statistics for every file currently active in the table state.¹⁶ This yields mathematically precise telemetry on row counts, byte sizes, and partition mutations with near-zero computational overhead, bypassing the Spark engine entirely.¹⁴

Using this parsed data, the agent calculates the Mutation Velocity (v_m) as the rate of change in row volume over a defined sliding temporal window:

$$v_m = \frac{\sum |\Delta \text{Rows}_{\text{added}}| + \sum |\Delta \text{Rows}_{\text{deleted}}|}{\text{Total Table Rows} \cdot \Delta t}$$

2.2 Thresholds for Statistical Decay and "Staleness"

Relying on the continuous calculation of mutation velocity, the agent defines an exponential decay model for statistical confidence. Parquet files are typically two to ten times smaller on disk than in memory due to highly efficient dictionary encoding and compression codecs like ZSTD or Snappy.⁴ When data mutates and statistics become stale, Catalyst is forced to rely on raw file-size heuristics, routinely underestimating the true memory footprint.⁴ Each estimation error compounds through the physical plan; a 2x error on filter selectivity combined with a 3x error on join cardinality produces a massive 6x estimation error in the final stage.⁴

The agent actively flags existing catalog statistics as "stale" or "decayed" when the Confidence Score $C(t)$ drops below an acceptable lower bound limit. The temporal decay is modeled as:

$$C(t) = C_{initial} \cdot e^{-\lambda \cdot \left(\int_{t_0}^t v_m(x) dx \right)}$$

Where:

- $C_{initial}$ is the confidence baseline immediately after a successful ANALYZE operation (normalized to 1.0).
- λ is an aggressive decay constant tied to the table's specific query sensitivity and SLA requirements.
- The integral represents the cumulative mutated fraction of the table since the last statistics update timestamp t_0 .

Actionable Decay Thresholds:

1. **Append-Only Time-Series Tables:** If the cumulative mutation fraction exceeds 15% of the total row volume, the statistics are flagged for a lightweight table-level update (COMPUTE STATISTICS without the FOR COLUMNS clause).³ This updates the raw byte sizes and row counts, restoring baseline broadcast capabilities.
2. **Highly Updated/Deleted Operational Tables:** When tables are subject to heavy GDPR deletions or continuous upserts via MERGE commands, Deletion Vectors and file updates cause rapid data fragmentation and write amplification.¹¹ A cumulative mutation fraction of just 5% immediately flags the table for a full, intensive column-level update (COMPUTE STATISTICS FOR COLUMNS).³ Furthermore, the agent couples this update with a compaction command (OPTIMIZE) to consolidate the fragmented small files, which resets the data skipping indices and drastically improves read performance.⁸

2.3 Hybrid Storage Environments: Static Partitions vs. Rapid Streams

Enterprise data platforms frequently feature highly complex hybrid storage environments. A ubiquitous pattern involves querying a mix of static, immutable historical Parquet partitions (e.g., archived data partitioned by year and month) alongside a rapidly mutating upstream Delta Lake stream capturing real-time operational events.¹⁸

Treating this hybrid architecture as a monolithic entity for statistics gathering leads to catastrophic resource waste. The agent is specifically designed to be partition-aware and storage-context sensitive to handle this dichotomy effectively.

Agent Intervention Strategy in Hybrid Modes:

- **Immutable Historical Partitions:** The agent recognizes the immutability of historical partitions by parsing the absence of transaction log activity over long temporal windows.⁵ For these deep storage tiers, the agent prescribes a one-time, highly intensive `ANALYZE TABLE... COMPUTE STATISTICS FOR ALL COLUMNS`.³ It further enhances this by triggering Liquid Clustering (`CLUSTER BY`) or Z-Ordering on the most frequent join keys, tightly physically co-locating the data to maximize file-level statistics pruning.¹¹ Once successfully processed, the agent's telemetry marks these partitions with a permanent decay constant of $\lambda = 0$, ensuring no future compute budgets are ever wasted re-analyzing immutable data.
- **Rapidly Mutating Streams:** For the "hot" streaming partitions undergoing micro-batch ingestion, the agent actively disables intensive column-level statistics gathering. Because the data mutates significantly faster than the statistics can be accurately utilized by the CBO, the agent dynamically sets the `delta.dataSkippingNumIndexedCols` to 0 or restricts it entirely to a single temporal event-time key.¹¹ The agent relies completely on AQE and the dynamic `spark.sql.cbo.enabled=false` context configuration for any queries hitting *only* the streaming partitions, bypassing the Catalyst CBO entirely to prioritize raw ingestion throughput and avoid unnecessary overhead.¹²

3. Execution Safety Guards & Guardrails

An autonomous AI agent granted the administrative authority to execute arbitrary database statistics commands poses a severe systemic risk to the platform. If misconfigured, the agent could inadvertently schedule massive metadata scans across billions of files, creating a catastrophic "denial of service" (DoS) condition on the Spark driver, the Hive Metastore, or the underlying cloud object storage layer. Consequently, programmatic execution safety guardrails are entirely non-negotiable within the architecture.

3.1 Object Storage Rate Limiting and Footer-Skimming Protections

When executing an `ANALYZE` command or evaluating queries over thousands of Parquet files, Spark workers must open the files and read the metadata footers to extract schema definitions, compression codecs, and row-group statistics.¹⁰ Cloud object stores, such as Amazon S3, impose strict API rate limits to protect their multitenant infrastructure. Specifically, S3 supports a maximum request rate of 3,500 `PUT/COPY/POST/DELETE` requests and 5,500 `GET/HEAD` requests per second, per partitioned prefix.²⁰

If the agent schedules a mass metadata listing or an `ANALYZE` job across a highly partitioned table containing a massive proliferation of small files, thousands of concurrent Spark tasks will request Parquet footers simultaneously. This will instantly exceed the 5,500 `GET` requests limit, triggering aggressive 503 Slow Down `AmazonS3Exceptions`.²¹ Task failures cause Spark to retry

the stages, further exacerbating the S3 backend overload in a cascading failure loop.¹⁰

Programmatic Guardrails for API Protection:

1. **Pre-Flight File Count Verification:** Before issuing any ANALYZE command, the agent queries the Delta transaction log using the DeltaTable.files() API to determine the total active file count.¹⁶ If the file count exceeds a predefined safety threshold (e.g., > 10,000 active files), the agent *hard-blocks* the statistics gathering. It instead autonomously triggers an OPTIMIZE (compaction) job utilizing autoCompact strategies to bin-pack small files into target 128MB or 256MB sizes.⁸ Only once the file count is reduced does it proceed with analysis.
2. **Concurrency Throttling Mechanisms:** The agent dynamically injects localized Spark configurations to artificially throttle the parallel execution of the ANALYZE job. By deliberately lowering spark.default.parallelism and drastically reducing the number of executor cores allocated specifically for the metadata job, the agent forces the S3 GET requests to remain safely beneath the 5,500 per prefix limit, ensuring natural load distribution across S3 partitions.¹⁰
3. **Exponential Backoff Configurations:** The agent automatically appends resilient I/O configurations to its orchestration context. By explicitly setting the fs.s3.maxRetries parameter to a high value (e.g., 20 or 30) within the EMRFS or equivalent filesystem client²², the agent ensures that if momentary throttling does occur, the Spark job backs off gracefully with exponential delays rather than crashing the entire application.²²

3.2 Catalog and Driver Denial of Service Prevention

Executing COMPUTE STATISTICS FOR ALL COLUMNS on exceptionally wide tables (e.g., machine learning feature stores with thousands of columns) creates massive JSON metadata payloads in the transaction log.³ The Spark query planner must load these statistics entirely into the driver's memory during the logical planning phase to enumerate join costs.¹¹ Excessive metadata directly causes prolonged planning times, severe Driver Memory OOMs, and query timeouts.¹¹

Programmatic Guardrails for Driver Protection:

- **Absolute Column Cap Limits:** The agent enforces a strict internal programmatic policy that restricts targeted column-level statistics gathering to a maximum of 50 to 60 columns per table, regardless of the schema size.¹¹
- **Predicate-Driven Target Selection:** The agent continuously parses historical logical query plans to extract the exact Filter and Join node expressions. It populates the delta.dataSkippingStatsColumns property exclusively with these high-value, highly selective predicate columns.¹¹ If a column is demonstrably never used in a WHERE or JOIN clause across the historical query profile, the agent explicitly blocks statistics generation for it. This surgical precision preserves critical driver memory, minimizes transaction log bloat, and vastly reduces the write amplification associated with maintaining statistics during ETL loads.¹¹

3.3 Handling Failure Modes and Cluster Preemption

The autonomous agent is designed to operate in highly dynamic, ephemeral distributed compute environments where hardware failures, network partitions, and resource preemptions are guaranteed to occur.

Failure Mode: Corrupt Transaction Logs and Legacy Checkpoints When interacting with legacy v1 changelogs, migrating between Iceberg and Delta formats, or attempting to read corrupted checkpoint files, Spark can occasionally throw fatal parsing errors such as `NumberFormatException` during the query planning phase.²⁴

- *Agent Handling Strategy:* The agent strictly isolates its planning and intervention validations inside ephemeral, isolated `SparkSessions`. If a `NumberFormatException` or similar metadata corruption error is caught during a trial physical plan generation, the agent immediately flags the table as "corrupted metadata." It halts all further optimization attempts and executes a safe fallback protocol: dynamically injecting `spark.sql.cbo.enabled=false`¹² for all subsequent production queries referencing that table. This forces the Spark engine to rely strictly on rule-based heuristics and AQE until data platform engineers can manually execute an FSCK or repair the transaction log.

Failure Mode: Cloud Resource Preemption ANALYZE and OPTIMIZE jobs are computationally heavy and often long-running. To optimize cloud compute spend, organizations typically schedule these maintenance workloads on AWS Spot Instances or preemptible Kubernetes nodes.²⁵

- *Agent Handling Strategy:* The agent ensures all operations are strictly idempotent. It configures advanced table properties such as `delta.checkpoint.writeStatsAsJson=true` and `delta.checkpoint.writeStatsAsStruct=true`²⁶ so that successfully processed file statistics are deeply integrated into intermediate checkpoints. If a Spot node is preempted midway through a massive analysis job, the agent catches the underlying YARN or K8s container kill signal exception.²⁵ It then implements a circuit breaker, logs the partial completion state, and autonomously reschedules the job with a highly restricted WHERE predicate. This ensures the retry processes only the remaining un-analyzed partitions, avoiding duplicated computational effort and spiraling costs.

4. Context Architecture and Technical Blueprints

To achieve this level of operational autonomy, the agent relies on an extensive, multi-dimensional context state. This state must encapsulate engine configurations, historical query profiles, temporal mutation metrics, and physical infrastructure topography. Below are the concrete JSON context schema extensions that represent the environment the agent monitors, followed by the Python-based architectural pseudocode mapping out the safe orchestration loop.

4.1 Schema Extensions: Comprehensive Context State

JSON

```
{  
  "agent_context_schema"  
  "target_table" "catalog.warehouse.fact_global_transactions"  
  "engine_context"  
  "spark_version" "3.5.0"  
  "aqe_enabled" true  
  "cbo_enabled" true  
  "auto_broadcast_threshold_bytes" 31457280  
  "parquet_compression_codec" "ZSTD"  
  
  "catalog_schema_state"  
  "total_columns" 250  
  "partition_columns" "region" "event_date"  
  "clustering_keys" "merchant_id"  
  "existing_stats_columns" "event_date" "merchant_id"  
  
  "temporal_context"  
  "last_analyzed_timestamp" "2026-05-18T10:00:00Z"  
  "mutation_velocity"  
  "rows_added_per_hour" 1500000  
  "rows_deleted_per_hour" 50000  
  "cumulative_mutation_fraction" 0.18  
  
  "decay_status" "CRITICAL_STALE"  
  "hybrid_storage_profile"  
  "static_partitions_predicate" "event_date <= '2025-12-31'"  
  "streaming_partitions_predicate" "event_date >= '2026-01-01'"  
  
  "infrastructure_context"  
  "cluster_topology"  
  "total_executor_cores" 128  
  "driver_memory_gb" 32  
  "storage_backend" "s3"  
  "s3_prefix_concurrency_limit" 5500  
  "spot_instance_ratio" 0.8  
}
```

```

"compute_budget"
"max_analyze_duration_sec" 1800
"cost_per_core_hour_usd" 0.045
"intervention_threshold_score" 75.0
"
"
"historical_context"
"query_execution_profiles"
"frequent_join_keys" "customer_id" "merchant_id"
"frequent_filter_predicates" "transaction_amount" "event_date" "status_code"
"observed_oom_events_last_7d" 2
"avg_query_latency_sec" 45.2
"p99_query_latency_sec" 180.5
"
"
"
"

```

This extensive JSON schema provides the autonomous agent with the exact inputs required to mathematically evaluate the Cost-Benefit Arbitration Matrix. The `historical_context` directly informs which specific columns will populate the `delta.dataSkippingStatsColumns` property, preventing driver OOMs.¹¹ Concurrently, the `infrastructure_context` dictates the precise throttling guardrails necessary to prevent S3 503 Slow Down events during footer skimming.²¹

4.2 Architectural Blueprints: Safe Orchestration Loop (Pseudocode)

The orchestration loop acts as the central nervous system of the agent. It ties the arbitration matrix, the temporal telemetry tracking, and the execution safety guardrails together into a continuous, resilient background process. The following Python-based pseudocode outlines the structural implementation of this control loop.

```

Python
class AutonomousSparkOptimizationAgent:
    def __init__(self, catalog_client, spark_session, context_schema):
        self.catalog = catalog_client
        self.spark = spark_session
        self.context = context_schema
        self.max_s3_retries = 30 # Guardrail for S3 503 SlowDown [23]
        self.max_stats_columns = 50 # Guardrail for Driver OOM

```

```

def orchestrate_table_maintenance(self, table_name):
    try
        # Phase 1: Temporal Telemetry and Decay Evaluation
        mutation_fraction = self.calculate_mutation_fraction(table_name)
        if mutation_fraction > 0.05 and self.context.temporal_context.decay_status !=
"CRITICAL_STALE":
            self.log_action("Table statistics within acceptable confidence intervals. Deferring
action.")
            return "HEALTHY_STATE"

        # Phase 2: Execution Safety Guardrails (DoS Prevention)
        active_file_count = self.get_active_file_count_via_delta_rs(table_name) # Avoid
Spark for metadata [17]
        if active_file_count > 10000:
            self.trigger_compaction(table_name) # Run OPTIMIZE to bin-pack small files
            self.log_action("File count exceeded safe API limits. Compaction triggered to protect
Object Store.")
            return "COMPACTION_TRIGGERED"

        # Phase 3: Extract Predicates for Targeted Statistics (Schema Pruning)
        target_columns = self.merge_lists()
        self.context.historical_context.frequent_filter_predicates =
self.context.historical_context.frequent_join_keys =
        # Enforce hard limits to protect driver memory
        safe_target_columns = self.enforce_column_limits(target_columns)
        max_cols = self.max_stats_columns

        # Inject targeted properties to override the default 32-column heuristic
        self.apply_table_properties(table_name, safe_target_columns)

        # Phase 4: Cost-Benefit Arbitration Matrix Evaluation
        intervention_score = self.evaluate_moo_arbitration_matrix(
            table_name=table_name,
            target_columns=safe_target_columns,
            file_count=active_file_count,
        )

        if intervention_score >
self.context.infrastructure_context.compute_budget.intervention_threshold_score:
            # Phase 5: Safe Execution with Infrastructure Throttling

```

```

        # Apply Exponential Backoff for S3 Resiliency
        self.spark.conf.set "fs.s3.maxRetries" self.max_s3_retries

    # Throttle S3 GETs by limiting parallelism
    self.spark.conf.set "spark.default.parallelism" 50

    query = f"ANALYZE TABLE {table_name} COMPUTE STATISTICS FOR COLUMNS
    {','.join(safe_target_columns)}"
    self.spark.sql(query)

    self.update_telemetry_statetable_name status "FRESH"
    self.log_action "CBO statistics successfully updated via targeted intervention."
    return "STATS_UPDATED"
else
    self.log_action "Arbitration matrix determined cost of ANALYZE outweighs query
    benefit. Deferring to AQE."
    return "DEFERRED_TO_AQE"

except Exception as e
    if "NumberFormatException" in str(e):
        # Handle Corrupt Transaction Logs or Checkpoints
        self.spark.conf.set "spark.sql.cbo.enabled" "false"
        self.log_action "Corrupt transaction log detected. CBO disabled; AQE fallback engaged
        for safety."
        return "METADATA_CORRUPTION_FALLBACK"

    elif "InterruptedException" in str(e) or "ExecutorLostFailure" in str(e):
        # Handle YARN/K8s Spot Node Preemption
        self.log_partial_completion_statetable_name
        self.trip_circuit_breaker()
        self.log_action "Agent preempted by cluster manager. State saved via JSON
        checkpoints for idempotent retry."
        return "PREEMPTED_RESCHEDULED"
    else
        raise

```

This structural loop ensures the agent behaves as a non-destructive, highly resilient background service. It proactively calculates statistical drift, rigorously checks the infrastructure compute budget, surgically mitigates object-store hammering through API throttling, and executes analytical queries with precise driver-memory isolation.

Synthesis and Architectural Conclusion

The architectural blueprints detailed in this analysis transform traditional static query optimization into an intelligent, autonomic, and highly resilient system. By synthesizing a theoretically sound Multi-Objective Cost-Benefit Arbitration Matrix⁷ with deep, non-invasive metadata telemetry extracted from the lakehouse transaction logs⁵, the autonomous agent actively navigates the complex tradeoff between computational analysis overhead and downstream query execution efficiency.

Crucially, the agent's design philosophy does not seek to deprecate or replace Adaptive Query Execution (AQE). Instead, it rigorously maps the boundary rules where AQE fundamentally fails—such as catastrophic out-of-memory errors triggered by massive initial un-shuffled datasets, or the optimizer's inability to dynamically re-plan deeply nested, multi-table star schemas post-execution. By explicitly injecting highly targeted column statistics³ via controlled, rate-limited execution loops that deeply respect S3 API thresholds²¹ and Spark driver memory constraints¹¹, the agent ensures the Catalyst CBO receives precise, actionable context before the first stage is ever submitted. The resulting ecosystem guarantees minimal operational waste, robust failure isolation against corrupted metadata and preemptible infrastructure²⁴, and maximum throughput for the most demanding large-scale distributed data workloads.

Works cited

1. 100 Scala Interview Questions and Answers for Data Engineers - DEV Community, accessed May 20, 2026, https://dev.to/hannah_usmedynska/100-scala-interview-questions-and-answers-for-data-engineers-4fd6
2. Spark SQL and DataFrame Programming Overview - NVIDIA, accessed May 20, 2026, <https://www.nvidia.com/pt-br/ai-data-science/spark-ebook/spark-sql-dataframes/>
3. What Spark Knows About Your Data: Statistics and the Cost-Based Optimizer | sparklearning, accessed May 20, 2026, https://vinodkc.github.io/sparklearning/catalyst/statistics_and_cbo.html
4. Spark SQL Join Strategy: The Complete Optimization Guide | Cazpian Docs, accessed May 20, 2026, <https://cazpian.ai/blog/spark-sql-join-strategy-the-complete-optimization-guide>
5. Comparison of Data Lake Table Formats (Apache Iceberg, Apache Hudi and Delta Lake), accessed May 20, 2026, <https://www.dremio.com/blog/comparison-of-data-lake-table-formats-apache-iceberg-apache-hudi-and-delta-lake/>
6. Comprehensive Guide to Optimize Databricks, Spark and Delta Lake Workloads, accessed May 20, 2026, <https://www.databricks.com/discover/pages/optimize-data-workloads-guide>
7. A Spark Optimizer for Adaptive, Fine-Grained Parameter Tuning - arXiv, accessed May 20, 2026, <https://arxiv.org/html/2403.00995v3>
8. Optimizations - the Delta Lake documentation, accessed May 20, 2026,

- <https://docs.delta.io/optimizations-oss/>
9. Cost Based Optimizer in Apache Spark 2.2 | Databricks Blog, accessed May 20, 2026,
<https://www.databricks.com/blog/2017/08/31/cost-based-optimizer-in-apache-spark-2-2.html>
 10. The Ultimate Guide to File Sizing & Compression for Apache Iceberg on S3 | by Sanjeeb Panda | Medium, accessed May 20, 2026,
<https://medium.com/@sanjeebmeister/the-ultimate-guide-to-file-sizing-compression-for-apache-iceberg-on-s3-88a5e57939f0>
 11. Collecting Delta Stats for Columns Used in Filters... - Databricks ..., accessed May 20, 2026,
<https://community.databricks.com/t5/data-engineering/collecting-delta-stats-for-columns-used-in-filters-beyond/td-p/143208>
 12. OCI Dataflow SQL EndPoint — performance analysis with TPC-DS 3000 | by Mario Miola, accessed May 20, 2026,
<https://medium.com/@mario.miola/oci-dataflow-sql-endpoint-performance-analysis-with-tpc-ds-3000-374d86e5ae21>
 13. Lakehouse architectures and open formats: ACID, time travel, multi, accessed May 20, 2026, <https://www.hardwinsoftware.com/blog/?p=1151>
 14. Native Delta Lake Support in DuckDB, accessed May 20, 2026,
<https://duckdb.org/2024/06/10/delta>
 15. Get statistics metadata · Issue #2233 · delta-io/delta-rs - GitHub, accessed May 20, 2026, <https://github.com/delta-io/delta-rs/issues/2233>
 16. Usage — delta-rs documentation, accessed May 20, 2026,
<https://delta-io.github.io/delta-rs/python/usage.html>
 17. New features in the Python deltalake 0.7.0 release of delta-rs, accessed May 20, 2026, <https://delta.io/blog/2023-02-27-deltalake-070-release/>
 18. Mastering Delta Lake MERGE Performance: Why It Slows Down and How to Fix It - Databricks Community, accessed May 20, 2026,
<https://community.databricks.com/t5/technical-blog/mastering-delta-lake-merge-performance-why-it-slows-down-and-how/ba-p/156305>
 19. Demystifying Optimization with Data Skipping | by Krishna Khadka - Medium, accessed May 20, 2026,
<https://khadkakrishna.medium.com/delta-lake-optimization-with-data-skipping-c9d2e78a1720>
 20. Improve operational efficiencies of Apache Iceberg tables built on Amazon S3 data lakes, accessed May 20, 2026,
<https://aws.amazon.com/blogs/big-data/improve-operational-efficiencies-of-apache-iceberg-tables-built-on-amazon-s3-data-lakes/>
 21. Best practices to optimize data access performance from Amazon EMR and AWS Glue to Amazon S3 | AWS Big Data Blog, accessed May 20, 2026,
<https://aws.amazon.com/blogs/big-data/best-practices-to-optimize-data-access-performance-from-amazon-emr-and-aws-glue-to-amazon-s3/>
 22. Resolve HTTP 503 "Slow Down" AmazonS3Exception in Amazon EMR | AWS re:Post, accessed May 20, 2026,

- <https://repost.aws/knowledge-center/emr-s3-503-slow-down>
23. Retry Amazon S3 requests with EMRFS, accessed May 20, 2026,
<https://docs.aws.amazon.com/emr/latest/ReleaseGuide/emr-spark-emrfs-retry.html>
 24. Databricks Runtime 17.1 (EoS) | Databricks on Google Cloud, accessed May 20, 2026, <https://docs.databricks.com/gcp/en/release-notes/runtime/17.1>
 25. Hortonworks Data Platform, accessed May 20, 2026,
<http://www.statwks.com/wp-content/uploads/2018/11/hdp-release-notes-3.0.0.pdf>
 26. Delta Table Properties Reference, accessed May 20, 2026,
<https://docs.delta.io/table-properties/>